Problem Understanding & Objective:

The goal of this project was to design an end to end anomaly detection system for IoT sensor data. In industrial environments, sensors continuously monitor equipment health. Unusual readings (spikes or drifts) can indicate early signs of failure or maintenance needs. This project detects such anomalies automatically using two complementary approaches:

1. Isolation Forest (Unsupervised Machine Learning)
2. LSTM Autoencoder (Deep Learning – PyTorch)

Dataset used:

- Ambient Temperature System Failure from the Numenta Anomaly Benchmark (NAB).
- Contains real-world hourly temperature readings leading up to a system failure.

Data Preparation and Exploration:

The dataset contained ~7,200 readings with timestamps and temperature values.
Steps performed:

- Loaded CSV data, parsed timestamps, and sorted chronologically.
- Checked for missing values → handled using **forward and backward filling**.
- Conducted visual EDA showing a **gradual upward drift** before the failure event.
- Saved clean data in data/ambient_temperature_system_failure.csv.

A quick line plot revealed temperature stability for most of the period, followed by rising fluctuations — indicating potential failure patterns.

Feature Engineering:

To make the data more meaningful for modeling, several features were derived:

Feature	Description
Rolling Mean (24h)	Captures local temperature average
Rolling Std (24h)	Captures local volatility
Lag 1, Lag 2	Previous readings to preserve short-term memory
Trend, Seasonality	Extracted using seasonal_decompose()
Scaling	Normalized using StandardScaler

These engineered features help the models detect both short-term spikes and long-term drifts.
Processed dataset was stored as data/processed_temperature_features.csv

Model Design and Implementation: Two complementary anomaly detection approaches were used.

Model 1: Isolation Forest (Unsupervised ML)

- Implemented using scikit-learn.
- Works by randomly partitioning data — points easier to isolate are likely anomalies.
- Configurations:
  - n_estimators = 200
  - contamination = 0.01 (1% anomalies expected)

- Output: Anomaly score and binary anomaly flag (1 = anomaly).

Visualization of anomalies over time clearly showed sharp spikes near the final failure period.

Model 2: LSTM Autoencoder (Deep Learning – PyTorch)

- Built a custom LSTM Autoencoder with encoder-decoder architecture.
- The model learns to reconstruct normal sequences of length 24 hours.
- If the reconstruction error is high, it indicates an anomaly.

Architecture Details:

- Input sequence length = 24
- Hidden/Embedding size = 32
- Optimizer: Adam | Learning Rate = 0.001
- Loss: Mean Squared Error (MSE)
- Epochs: 20
- Hardware: GPU (CUDA) enabled system

Threshold used:

mean + 3 × standard deviation of reconstruction error.

Sequences exceeding this threshold were flagged as anomalies.

Model Evaluation:

Since the dataset does not contain labeled anomalies, evaluation relied on:

- **Visual inspection** of anomaly regions.
- **Overlap comparison** between Isolation Forest and LSTM Autoencoder.
- **Correlation** between anomaly scores and reconstruction errors.

Metric	Isolation Forest	LSTM Autoencoder
Total Anomalies	73	226
Common Anomalies	53	-
Overlap (%)	23.4%	-

Observations:

- **Isolation Forest**: Detected sudden spikes or extreme readings.

- **LSTM Autoencoder**: Identified gradual drift patterns over time.
- Overlapping anomalies (23%) are high-confidence points representing actual abnormal behavior.

Key Results and Insights

1. Isolation Forest provided quick and interpretable results.
2. LSTM Autoencoder learned complex temporal dependencies and performed better in detecting subtle trends.
3. Combining both methods yields robust anomaly coverage.
4. Clear visualization plots confirm anomalies appear near system failure.

Outputs generated:

- isolation_forest_results.csv
- lstm_autoencoder_results.csv,
- comparison_isoforest_lstm.png

Limitations and Future Scope

Area	Limitation	Future Improvement
Dataset	Univariate (temperature only)	Extend to multivariate (vibration, pressure, humidity)
Threshold	Static (3σ rule)	Use adaptive MAD or percentile thresholds
Model Tuning	Basic hyperparameters	Add Bayesian or grid search tuning
Real-time Capability	Batch mode	Integrate streaming with Flask/FastAPI dashboard

Business Insights

- Real-time anomaly alerts can **prevent costly equipment failures**.
- Trend-based drift detection allows **predictive maintenance**.
- Modular pipeline allows scaling across multiple IoT devices.

This project provides a foundation for production-ready predictive maintenance systems.

Conclusion

This project uses ML pipeline with both Isolation Forest and LSTM Autoencoder models. It is well-documented, visually explained, and designed to be easily deployable for real-world IoT anomaly detection and monitoring systems.